

A Secure Identifier-to-Locator Mapping Mechanism in Smart Identifier Network

Jiahui Sun*, Ningchun Liu*, Shuai Gao^{*†}, Wei Su*

*School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing, China

[†]PCL Research Center of Networks and Communications, Peng Cheng Laboratory, Shenzhen, China

Email: {jiahuisun,nchliu,shgao,wsu}@bjtu.edu.cn

Abstract—In recent years, the rapid development of Programmable Data Plane(PDP) technology provides the possibility for the large-scale deployment of Smart Identifier Network(SINET). As a revolutionary network architecture, SINET supports terminal mobility and network extensibility natively by adopting the separation of identifier and locator. At the beginning of the SINET design, the security of the mapping information in the mapping publish/query protocol was not considered. At the same time, because of the PDP-based SINET using a separate control and forwarding architecture, the mapping information between the control plane and the data plane is more vulnerable to eavesdropping attacks and man-in-the-middle attacks.

In this paper, we proposed a secure Identifier-to-Locator mapping mechanism in the PDP-based SINET. We designed a bCPA(bit-wise CPA) algorithm to protect the mapping information in the mapping publish/query protocol. We implemented the mechanism and evaluated its performance in the prototype system. The experimental results show that the method we designed guarantees the security of the mapping information without affecting the mapping publish/query performance and has less time increment compared to existing methods.

Index Terms—SINET, PDP, security, secret permutation

I. INTRODUCTION

The traditional IP addresses are semantically overloaded since they identify both hosts and topological locations. Researches on next-generation Internet architecture become a hot topic. As a representative architecture, SINET decouples terminal identifier and locator using a mapping mechanism [1]. To some extent, this architecture is a viable solution to the core network security and extensibility.

However, as a disruptive Internet architecture, it redefines the Internet packet and the way of forwarding, which needs extensive modifications of all equipments in the network. Because of the high cost of modifications, the deployment of SINET remains slow and only initial deployment attempts were made in industry-specific networks. In recent years, PDP technology, represented by P4 [2], has been booming. Different from traditional technologies, which are limited to a specific kernel stack type, P4 provides a programmable packet forwarding model that can be customised to encapsulate arbitrary network layer packet headers, providing an opportunity for SINET further deployment.

Since the original design of SINET was limited to the separation of identifier and locator and did not involve mapping information protection, which poses a challenge to network

security. Specifically, on one hand, once AID(Access Identifier) and RID(Route Identifier) containing terminal identity and location information are leaked or forged, an attacker can use the forged AID to interfere with the normal communication. On the other hand, in mapping publish protocol, ASR(Access Switch Router) needs to publish the mapping information to the mapping server. An attacker can easily intercept mapping information $\langle AID, RID \rangle$ by eavesdropping attacks and man-in-the-middle attacks(MITM).

These attacks become more popular in P4-based SINET because of the separation of control plane and data plane. On one hand, P4 aims at programmable packet forwarding rather than security and its equipments have limited computing and storage resources to support the complex power operations brought by the traditional public key system and signature mechanism. Therefore, an effective and lightweight encoding mechanism needs to be designed in SINET to verify the safety of mapping information. On the other hand, because of the separation of the control plane and the data plane, SINET are vulnerable to security threats from MITM attacks and eavesdropping attacks. So how to protect the security of mapping information from possible attacks is worth considering.

Based on the above problems, we propose a Secure Identifier-to-Locator Mapping Mechanism and design bCPA to protect the mapping information in the mapping publish/query protocol. This mechanism encrypts $\langle AID, RID \rangle$ before delivering packets between control plane and data plane and can decode the cipher text using the same key. The algorithm we design is faster than the existed algorithms and with little affect to the system.

The rest of the paper is organized as follows. Section II discusses related works of SDN security. Section III proposes our SINET security architecture based on P4. Section IV introduces the Secure Identifier-to-Locator Mapping Mechanism. Section V gives a brief security analysis of the architecture. We evaluate our architecture in terms of both time and space overheads in Section VI. Section VII is the summary of our paper.

II. RELATED WORK

Because of the vulnerable controller and risks caused by open programmable interfaces, security threats attract more and more attentions from manufacturers and operators

TABLE I
THE SOLUTIONS OF FLOW RULES SECURITY

schemes	Changes in the architecture	location	Data Plane
Extended TLS	Add timestamp in TLS	Southbound protocol	OpenFlow
Source Encryption	Add another encryption before TLS	Control plane	OpenFlow
SecFT-SDN	A flow table security framework	Control plane	OpenFlow
Dynamically monitor anomaly	A way to monitor and defend attacks	Southbound link	OpenFlow
bCPA	Add an encoding mechanism in P4 switch	Southbound link	P4

[3]. Scholars have given possible solutions to this problem. S.Midha figures out that these attacks are responsible due to negligence in the handshake process where client/server authentication is missed or remains optional [4]. In order to make OpenFlow communication more secure, they proposes extended TLS support and defensive algorithm [5]. It adds a timestamp exchange action in the authentication process for generating and verifying the identity of either client or server. This idea is an addition to the existing TLS support. The disadvantage of this method is that it needs to update TLS protocol in OpenFlow from time to time.

TLS also has its defects. When there is an intermediate device such as a firewall between the controller and the switch, the TLS protocol states that it will be exposed in plain text at the intermediate device. This increases the risk of information leakage. Y.Wang add a source encryption before executing TLS using Kerberos authentication [6]. This ensures at least one encryption exists in the packet delivery process but make packet processing more redundant.

R.You starts from the controller and implements a flow table security framework, named as SecFT-SDN, on ONOS, an open source SDN controller [7]. SecFT-SDN installs flow rule test set to enhance the security protection facilities on ONOS controller, while incurs an acceptable overhead as a cost-effective trade-off. But the framework cannot support the secure problem in current P4 environment as the southbound protocol is P4 Runtime.

When it comes to the data plane, lack of security mechanisms also threats SDN switch. Y.Qian figures the possible attacks in data plane and find a new approach to dynamically monitor anomaly of flow table growth [8]. If the filling speed exceeds a predefined threshold, it is regarded as suspicion and triggers the defense actions to drop the malicious rules. This can militate the security threats to the flow table channel between switches and controller.

The current researches on P4-based security mechanisms focus on secure packet forwarding on the data plane. YB.Lin proposed an enhanced content permutation algorithm (eCPA) that can effectively implement secret permutation in a 5G/IoT transport network with the P4 switches guaranteeing the security of packet forwarding on the data plane while improving the transmission efficiency of the data plane [9]. But does not cover the mapping publish process in SINET.

III. PROPOSED ARCHITECTURE

A. Overview and Assumptions

In this paper, we propose a P4-based Secure Identifier-to-Locator Mapping Mechanism in Smart Identifier Network, which combines content permutation techniques to encrypt $\langle AID, RID \rangle$ in the southbound interface in a lightweight manner, effectively safeguarding the confidentiality of mapping information.

Fig. 1 is an overview of the whole architecture. We use ONOS as the controller, connecting the upper level application through northbound protocol and communicate with P4 switches using P4Runtime. Data Plane is divided into the core network and the access network. We use AID in access network and RID in core network. ASRs (Access Switch Routers) are placed at the junction of the access and core networks. They are responsible for identifier conversion and interacting with the control plane.

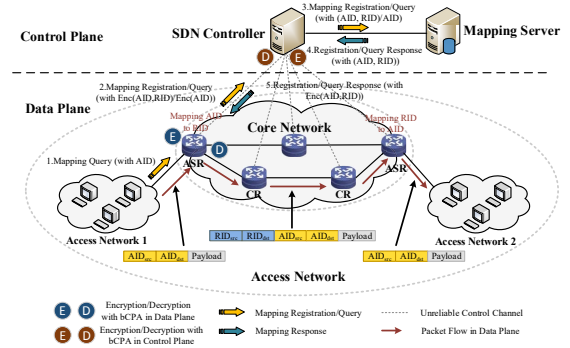


Fig. 1. the architecture overview

Our architecture makes appropriate additions to the SINET architecture. On the data plane, ASR encrypts mapping information using specific key before sending packets to the controller. On the control plane, APP can decode it with the same key following a similar rule. The cipher key is only stored in ASR and the controller, so it can prevent mapping information leakage due to man-in-the-middle attack.

As the P4 switch does not have CA certificates or other complex functions and can only perform simple actions based on the P4, the key we use for encryption in this mechanism will be configured in the controller and the ASRs off-line. This ensures the security of the key and has negligible impact on

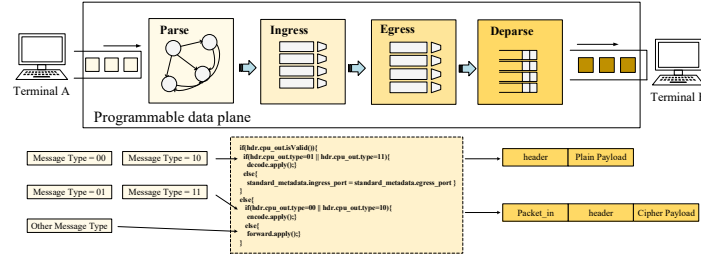


Fig. 2. data plane flow of the secure identifier-to-locator mapping mechanism

the overall architecture. So, we assume that the key has been configured off-line in advance.

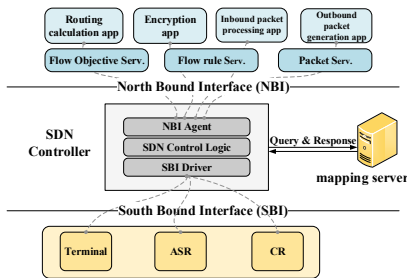


Fig. 3. control plane flow of the secure identifier-to-locator mapping mechanism

B. Data Plane

The data plane is divided into core network and access network. Terminals are only deployed in access network and switches are in core network. There are two types of switches in core network, ASR and CR(Core Router). ASR is located at the edge of core network and access network and is responsible for AID check and identifier mapping. If ASR is in charge of the destination AID, it will forward the packet directly to the terminal. Otherwise it will add RID to the packet and send it to core network. CR in core network forwards the packet according to RID. The actions of switches are directed by P4. Fig. 2 illustrates the data plane flow.

The data plane contains communication, publish and query protocol. Publish is responsible for storing $\langle AID, RID \rangle$ relationships into mapping server. Query is responsible for getting RID corresponding to AID. Then it can use RID to communicate in core network.

C. Control Plane

The control plane consists of the controller and upper APPs for routing logic generation. It can communicate with the data plane through P4Runtime. Fig.3 shows the control plane flow based on ONOS.

When the controller needs to distribute messages, the Routing Calculation APP calculates the weight of each link along the path between the receiver and the sender and select the optimal one according to weighted Dijkstra algorithm.

Before encapsulate the message in Packet_out, the Encryption APP use CPA to encrypt information, guaranteeing the safe transmission through P4Runtime. Then the Outbound packet generation APP will pack the data into Packet_out and deliver it to the switch on data plane.

IV. A SECURE IDENTIFIER-TO-LOCATOR MAPPING MECHANISM DESIGN

Fig.4 demonstrates the working flow of our architecture. The working flow of SINET is divided into three stages: registration, query and message transmission. We designed bCPA to suit the P4 environment. We begin here by stating that publish is executed after access authentication. Due to space limitations, access authentication will not be covered in depth here. In this paper, we assume AIDs are legitimate.

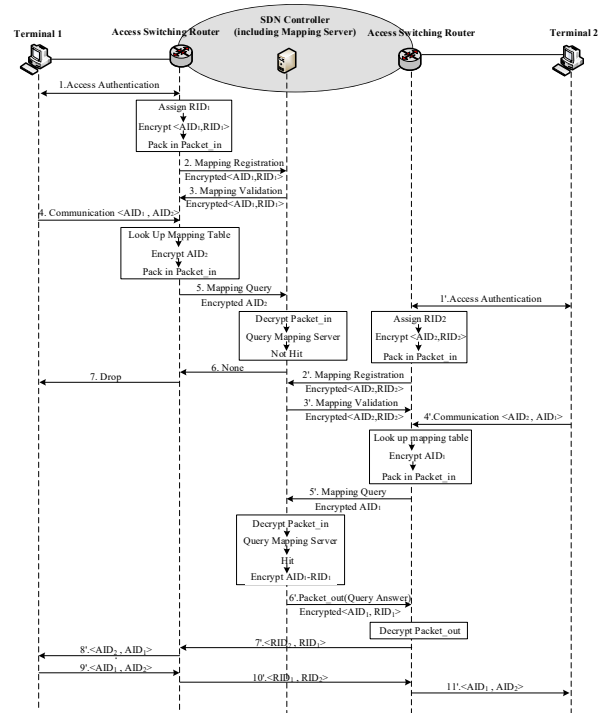


Fig. 4. working flow of the secure identifier-to-locator mapping mechanism

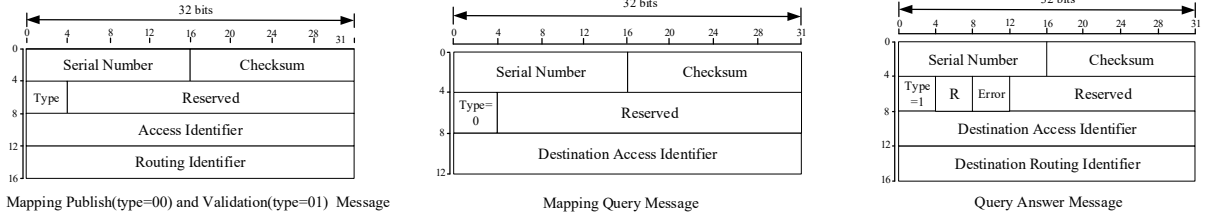


Fig. 5. publish protocol(nexthdr=3)and query protocol(nexthdr=4)

A. mapping publish

When terminal1 first connects to ASR, it will send a authentication message to ASR with terminal AID_{src} . ASR will assign a RID_{src} to AID_{src} and send this publish message in Packet_in to the control plane. This relationship will be stored in both the repository in ASR and the mapping server on the control plane. The publish message and validation message are shown in Fig.5.

When the publish message arrives, the controller will first decode it using bCPA and store the relationship in the mapping server. Then it will return a validation message indicating Succeed or Fail. As the message contains $\langle AID, RID \rangle$ relationships, they should be encrypted before sending. The encode and decode method will be introduced later.

B. ASR mapping query Message

When Terminal 1 sends a message to Terminal 2, it will put AID_{src} and AID_{dst} in the header and send it to ASR. ASR will first check its repository according to AID_{src} and AID_{dst} . By convection, only the relationships of terminal directly connected to the ASR are stored in the repository. So we use mapping query message and query answer message to find the corresponding RID_{dst} . ASR sends mapping query message in Packet_in, which contains encrypted AID_{dst} for querying, towards the control plane and receive encrypted $\langle AID_{dst}, RID_{dst} \rangle$. The mapping query message and query answer message are shown in Fig.5.

The encoding principle of bCPA is shown in Algorithm1. For $n \geq 2$, we partition the field need to be encrypted into n codewords. Denote n codewords as $S_n = \langle \pi_1(0), \pi_2(0), \dots, \pi_n(0) \rangle$ where π_i is the i^{th} codeword of the field. $S_n(i) = \langle \pi_1(i), \pi_2(i), \dots, \pi_n(i) \rangle$ represents the result after i^{th} permutation. Denote a binary cypher vector of size n as $K_{n-1} = \langle x_1, x_2, \dots, x_{n-1} \rangle$, where $X_i \in \{0, 1\}$ for $1 \leq i \leq n$. bCPA permutes S_n with the key K_{n-1} and the output result is C_n . We use Buff to store the index similar to CPA. Line 1-11 are a loop that is executed $n-2$ times. We divide the encoding actions by key value. If $x_i = 1$, shift code words from $\pi_n(i), \pi_{n-1}(i), \dots, \pi_{i+1}(i)$ to $\pi_{n-1}(i-1), \pi_{n-2}(i-1), \dots, \pi_i(i-1)$. If $x_i = 0$, do not change the position of codeword i . If $i = n-1$, jump out of the loop and executes Line 13-19. Each codeword is finally located in a different position. So, in the

view of the complete packet, it is a set of disordered mess code.

In SINET, AID and RID are 32 bits. We encrypt the mapping information using the given key. The switches encode 64 bits: $\langle 32 \text{ bits } AID, 32 \text{ bits } RID \rangle$ and 32 bits AID. 64 bits fields are divided into two parts, each part is separated into 16 blocks. (You can choose the granularity of the blocks). 32 bits fields use only one part encode and is also separated into 16 blocks.

Algorithm 1: bCPA-based Encryption in Programmable Data-Plane

Input: Secret: $S_n = \langle \pi_1(0), \pi_2(0), \dots, \pi_n(0) \rangle = S_n(0)$ ($S_n(i) = \langle \pi_1(i), \pi_2(i), \dots, \pi_n(i) \rangle$, where $i = 0$),
 Encrypt Key: $K_{n-1} = \langle x_1, x_2, \dots, x_{n-1} \rangle$, and
 Buffer: Buf .

Output: Ciphertext: $C_n = \langle \gamma_1, \gamma_2, \dots, \gamma_n \rangle = S_n(n-1)$ ($S_n(i) = \langle \pi_1(i), \pi_2(i), \dots, \pi_n(i) \rangle$, where $i = n-1$).

```

1 for  $i = 1$  to  $n-2$  do
2   if  $x_i = 1$  then
3      $Buf = \pi_n(i-1)$ ;
4     for  $k = n-1$  to  $i$  do
5        $\pi_{k+1}(i) = \pi_k(i-1)$ ;
6     end
7      $\pi_i(i) = Buf$ ;
8   else
9     NoAction();
10  end
11 end
12 while  $i = n-1$  do
13   if  $x_i = 1$  then
14      $Buf = \pi_n(i-1)$ ;
15      $\pi_{i+1}(i) = \pi_i(i-1)$ ;
16      $\pi_i(i) = Buf$ ;
17   else
18     NoAction();
19   end
20 end
```

Algorithm 2: bCPA-based Decryption in Control-Plane

Input: Ciphertext: $C_n = \langle \gamma_1, \gamma_2, \dots, \gamma_n \rangle = S_n(n-1)(S_n(i) = \langle \pi_1(i), \pi_2(i), \dots, \pi_n(i) \rangle, \text{ where } i = n-1)$, Symmetric Key: $K_{n-1} = \langle x_1, x_2, \dots, x_{n-1} \rangle$, Buffer: Buf and Decrypt Index: $index$.

Output: Plaintext:

```

 $S_n^* = \langle \pi_1(0), \pi_2(0), \dots, \pi_n(0) \rangle = S_n^*(0)($ 
 $S_n^*(i) = \langle \pi_1^*(i), \pi_2^*(i), \dots, \pi_n^*(i) \rangle, \text{ where } i = 0).$ 
1  $max = index - 1, min = index + 1, \pi_{index}^* = \pi_n$  for
   $i = n - 1$  to 1 do
2   if  $x_i = 1$  then
3      $\pi_{max}^* = \pi_i;$ 
4      $max = max + 1;$ 
5   else
6      $\pi_{min}^* = \pi_i;$ 
7      $min = min - 1;$ 
8   end
9 end
10 end
```

C. Control Plane Receive mapping query message

When a Packet_in packet arrives at the controller, the Inbound Packet Processing APP will unwrap the header field and decrypt the 32 bits cipher field following eCPA rules in Algorithm 2 and get RID_{dst} .

The controller will permute 16 blocks with the same key in bCPA. The bCPA decryption algorithm manipulates an $index$ and two pointers: $max = index - 1$, $min = index + 1$. In each iteration, the key is used to determine whether codewords should be moved. If $x_i = 1$, the codeword pointed by max is placed into π_i^* and max indexes the next codeword. Otherwise, the codeword pointed by min is placed into π_i^* and min indexes the next codeword. For more details on the CPA algorithm, please refer to [9].

D. Control Plane send mapping answer message

After getting the answer of RID_{dst} , the controller will form a query answer message with encrypted $\langle AID_{dst}, RID_{dst} \rangle$. The controller encrypts 64 bits: $\langle 32\text{bits } AID_{dst}, 32\text{bits } RID_{dst} \rangle$, which is divided into 16 blocks and send the Packet_out through P4Runtime. The CPA decryption algorithm operates in the opposite way of encryption. In this way, the relationship between AID and RID cannot be exposed even if a third-party hijack the packet.

After decrypting the cipher text, the controller will look up the mapping server with AID_{dst} and form a query answer message containing the mapping result $\langle AID_{dst}, RID_{dst} \rangle$ and use eCPA to encrypt the result. Then send the Packet_in through P4Runtime.

The controller encrypts 64 bits: $\langle 32\text{ bits } AID_{dst}, 32\text{ bits } RID_{dst} \rangle$, which is divided into 2 parts and each part contains 16 blocks. The CPA encryption algorithm operates in the opposite way of decryption, so we do not repeat here.

In this way, the relationship between AID and RID cannot be exposed even if a third-party hijack the packet.

E. Data Plane receive mapping answer message

When the query answer message with encrypted $\langle AID_{dst}, RID_{dst} \rangle$ relationship arrives at ASR, ASR use bCPA decrypt 64 bits: $\langle 32\text{bits } AID_{dst}, 32\text{bits } RID_{dst} \rangle$. It follows the same decomposition method in controller and the reverse decrypting order in Algorithm 2. Then we use the plain $\langle AID_{dst}, RID_{dst} \rangle$ and add RID header to transmit in core network. The opposite process will execute when the packet moves out the core network and deliver the message using AID in another access network.

V. SECURITY ANALYSIS

In this section, we introduce the two main attacks exits in the packet delivery process. We analysis their threat and the way our mechanism addresses them.

A. Man-in-the-Middle Attack

Man-in-the-Middle attack aims at hijacking the packet from the controller and intercepting with information. In SINET, a middle device between controller and switch may cut off the publish or query message. Without bCPA, the attacker can store this relationship and send wrong messages to the destination.

In our architecture, the message cut off by the attacker contains cipher text. As the attacker does not have the key, he could not decrypt it into plain text. In this way, even if the packet is hijacked by a third-party, bCPA can guarantee the security of sensitive information and prevent user information from leakage.

B. Network eavesdropping attacks

An eavesdropping attack is an intrusion. The attacker tries to steal information transmitted over a network from a computer or other device. It is difficult to detect eavesdropping attacks because network transmissions may not appear to be operating abnormally.

In our architecture, a network eavesdropping attacker who wants to steal information about interactions between end-points needs to obtain relationships of all $\langle AID, RID \rangle$. The encryption method makes it impossible for attackers to obtain them because the lack of key. In this way, the eavesdropping attacks can be avoided.

VI. SIMULATION EXPERIMENTS AND ANALYSIS

A. Experimental Environment

This section evaluate the performance of the Secure Identifier-to-Locator Mapping Mechanism in this paper in terms of the time overhead and the effect to packet forwarding.

In the simulation, we set up a linear topology with two terminals and a P4 switch in the prototype system. The key is already configured in the switch and controller. The switch parses and encrypts the $\langle AID, RID \rangle$ field per 2 bits and the controller can decrypt it. ONOS of version 2.2 is used

as the controller and runs a custom APP to handle routing packets. Controller is run on the device with Intel(R) Xeon(R) Silver 4214 CPU 2.20GHz processors and 64G of RAM. The data plane uses mininet version 2.3.0b to build the network topology with the bmv2 (version 1.13.0) as the switch. This node runs on a device with an Intel(R) Xeon(R) CPU X3440 @ 2.53GHz and 16G of RAM. The kernel versions of the above devices are Linux version 4.4.0-197-generic build@lcy01-amd64-026, Ubuntu 16.04.

B. Simulation Result

We compare the encryption and decryption time overhead of bCPA and AES128, which is the most popular encryption method at present. There are two types of message that need encryption, which is 32bits and 64bits. We use 32bits here as an example and process in units of 2bits.

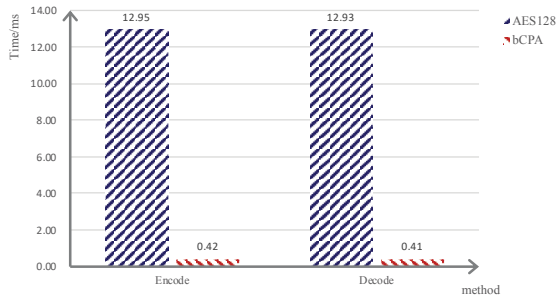


Fig. 6. encoding and decoding time of AES128 and bCPA

We evaluate the encoding and decoding time of bCPA and AES128 which is widely used. According to Fig.6, the overhead time of bCPA in P4 switch and CPA on the control plane has a decrease of 96.76% and 96.83% respectively comparing to AES128. This actively demonstrates that bCPA can save a lot of processing time compared to AES128. So bCPA can be seen as an efficient encryption algorithm and have a significant increase in encryption performance.

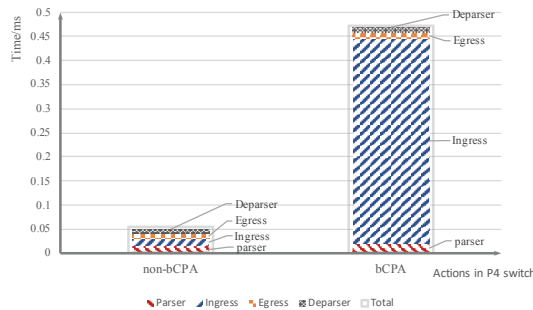


Fig. 7. packet processing time of P4 switch with and without bCPA

We also evaluate the whole packet processing time. In P4, the process is divided into four part. We compare the forwarding duration with and without bCPA. Fig.7 indicates

that the parser, egress and deparser has little increase because we need to parse more headers. Most increments come from ingress which contains bCPA. Note that the total time is longer than the sum of each part. This is caused by the addition of printing time to the log, which also have time consumption. The algorithm we add brings only 0.4 ms time increment but ensures security of user identity in SINET. For comparison, the lowest performance penalty in [7] is 7.17 ms, which is much higher than our architecture. Thus this time increment is considered acceptable.

VII. CONCLUSION

In this paper, we proposed a P4-based secure identifier-to-locator mapping mechanism and introduce a bCPA for user security. The encryption mechanism runs on both the ASR and the controller, making mapping relationships that interact between the data plane and the control plane unavailable to third parties. The prototype system is carried out to evaluate the time overhead of bCPA and the increment of the total forwarding time. The result illustrates that our architecture has a 96.79% decrement in encryption time and the time increment in packet forwarding is acceptable.

ACKNOWLEDGMENT

This work was supported by the National Key R&D Program of China (No. 2020YFB1806400), the National Nature Science Foundation of China (No. 61972026).

REFERENCES

- [1] H. Zhang, W. Quan, H.-c. Chao, and C. Qiao, "Smart identifier network: A collaborative architecture for the future internet," *IEEE network*, vol. 30, no. 3, pp. 46–51, 2016.
- [2] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.
- [3] Z. Shu, J. Wan, D. Li, J. Lin, A. V. Vasilakos, and M. Imran, "Security in software-defined networking: Threats and countermeasures," *Mobile Networks and Applications*, vol. 21, no. 5, pp. 764–776, 2016.
- [4] S. Midha and K. Triptahi, "Extended tls security and defensive algorithm in openflow sdn," in *2019 9th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, pp. 141–146, IEEE, 2019.
- [5] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "Openflow: enabling innovation in campus networks," *ACM SIGCOMM computer communication review*, vol. 38, no. 2, pp. 69–74, 2008.
- [6] Y. Wang, S. Zheng, L. Gu, and Y. Cai, "Source encryption scheme in sdn southbound," in *International Conference on Advanced Hybrid Information Processing*, pp. 560–569, Springer, 2018.
- [7] R. You, B. Tu, Z. Yuan, and J. Cheng, "Secft-sdn: Securing the flow-table for software-defined network," in *2019 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*, pp. 1139–1146, IEEE, 2019.
- [8] Y. Qian, P. Bhattacharya, W. You, and K. Qian, "Security threat analysis of sdn switch flow table," in *2018 27th International Conference on Computer Communication and Networks (ICCCN)*, pp. 1–2, IEEE, 2018.
- [9] Y.-B. Lin, T.-J. Huang, and S.-C. Tsai, "Enhancing 5g/iot transport security through content permutation," *IEEE Access*, vol. 7, pp. 94293–94299, 2019.